Razk Automation

Employee Management System

A professional attendance and HRMS application for employee management, attendance tracking, approvals, reports, and internal communication.

Prepared as a professional project explanation document

This document explains what was created, how the application works, the technologies used, and the major workflows available in the completed project.

Generated on 16 June 2026

1. Project Overview

Razk Automation Employee Management System is a full-stack HRMS and attendance application. It provides a secure web and mobile-ready interface for managing employees, attendance, leave and permission requests, on-duty requests, visitor visits, announcements, notifications, dashboards, and professional reports.

The application was created to reduce manual HR work and give Admin, HR, DRI, and Employee users a single controlled system for daily attendance and request management. It helps a business record attendance accurately, process approvals, communicate announcements, and export attendance reports.

Who Can Use the App

Role	Main Purpose
Admin	Manage the organization, employees, office settings, dashboards, reports, announcements, and approvals.
HR	Manage employees, attendance operations, requests, reports, visitor visits, announcements, and approvals assigned to HR.
Employee	Login, mark attendance, view personal dashboard, apply requests, view announcements, and manage profile.
DRI	Use employee-style attendance and request pages, plus assigned request workflows where applicable.

2. What Was Created in This Project

Frontend Pages and Screens

- Login, forgot password, and reset password screens.
- Role-based dashboards for Admin, HR, Employee, and DRI users.
- Employee list, add employee, edit employee, and employee profile screens.
- Attendance page with check-in, check-out, history, all-attendance view, and week-off handling.
- Attendance reports page with preview, PDF export, and Excel export.
- Leave, permission, OD request pages, and DRI assigned request screens.
- Announcements, visitor visits, profile, settings, notification bell, theme toggle, and shared layout components.
- Installable PWA support and Capacitor Android shell for mobile use.

Backend APIs Created

API Area	Purpose
/api/auth	Login, Admin/HR registration, user profile, password change, forgot password, and reset password.
/api/employees	Employee creation, listing, profile fetch, update, and deletion with role control.

API Area	Purpose
/api/attendance	Check-in, check-out, today status, personal history, all attendance, CSV export, and HR/Admin week-off marking.
/api/leave, /api/permission, /api/od	Request creation, listing, approval, rejection, and assigned workflow handling.
/api/reports	Employee, monthly, department, and custom attendance reports in preview, PDF, and Excel formats.
/api/announcements	Create, read, update, and delete announcements for Admin/HR, with role targeting.
/api/notifications	Unread notification list, counts, mark as read, mark all read, and deletion.
/api/uploads	Protected image and document upload endpoints.
/api/office-location	Office location records and distance preview for reference.
/api/visitors	Visitor visit creation, listing, update, and deletion for Admin/HR.
/api/dashboard	Admin, HR, and Employee dashboard data.
/api/health	Server, database, and environment health status.

Database Models and Collections

Model	Stores
User	Employee identity, login email, hashed password, role, employee ID, department, shift, weekly week-off day, profile data, active status, and password reset tokens.
Attendance	Daily employee attendance, check-in/out time, GPS details, shift name, working hours, status, week-off audit fields, and remarks.
LeaveRequest	Leave type, date range, reason, attachment, paid/unpaid split, approver details, status, and decision history.
PermissionRequest	Permission type, date, time range, reason, paid/unpaid hours, approver details, and decision history.
ODRequest	On-duty date, time, location, reason, attachment, approver details, and decision history.
Announcement	Title, message, creator, target role, and timestamps.
Notification	User notifications, title, message, type, read status, creator, and timestamps.
OfficeLocation	Office name, latitude, longitude, radius, and active/inactive status.

Model	Stores
Visitor	Visitor name, mobile number, company, purpose, person to meet, visit date, check-in/out, image, remarks, and audit users.

3. Main Functions of the App

Authentication and User Access

- Users login with email or employee ID and password.
- Passwords are hashed using bcrypt before storage.
- JWT tokens protect backend API access, and protected frontend routes redirect unauthenticated users to login.
- The app stores login state in browser session storage so users are asked to login again after closing and reopening the app.

Employee and Role Management

- Admin and HR users can add, view, update, and delete employees.
- Each employee can have a department, designation, assigned shift, weekly week-off day, profile photo, contact details, and active status.
- Roles control navigation menus, dashboards, API access, and approval capabilities.

Attendance Management

- Employees, HR, Admin, and DRI users can mark check-in and check-out when allowed.
- The system stores time, optional GPS coordinates, accuracy, and location status.
- Attendance statuses include Present, Absent, Half Day, Late, OD, Leave, Missed, and Week Off.
- Weekly Week Off is configurable per employee and can be any one day of the week.
- HR/Admin can mark Week Off only from Leave, Absent, or Missed status and the system prevents more than one Week Off in the same week.

Request and Approval Workflow

- Employees and DRI users can apply for leave, permission, and OD requests.
- Requests are assigned to HR/Admin approvers based on requester role and department rules.
- Approvers can approve or reject pending requests, while users cannot approve their own request.
- Approved/rejected decisions store actor name, role, remarks, and decision time.
- Notifications are created for assigned approvers and decision recipients.

Reports, Announcements, Notifications, Uploads, and Visitors

- Admin and HR can preview attendance reports by employee, month, department, or custom range.

- Reports can be exported as PDF using PDFKit and Excel using ExcelJS.
- Admin and HR can create, update, and delete announcements; employees can view targeted announcements.
- The notification bell shows unread notification counts and allows users to mark notifications as read.
- Protected upload APIs support images and documents using Multer.
- Admin and HR users can manage customer or visitor visit records.

4. App Workflow

User Flow

- A user opens the web app or installed PWA/mobile app and lands on the login page.
- The user enters email or employee ID and password.
- The frontend sends the login request to the Express backend API.
- The backend checks the user in MongoDB, compares the password using bcrypt, and returns a JWT token if valid.
- The frontend stores the token in session storage and redirects the user to the correct dashboard based on role.
- Protected screens call APIs with the JWT token in the Authorization header.
- The backend validates the token, checks the user role, processes the request, and reads or writes data in MongoDB.
- For requests and decisions, the backend updates request records and creates notifications for relevant users.
- For reports, the backend builds report data from users, attendance, and approved leave records, then returns JSON, PDF, or Excel output.

Role Flow After Login

Role	After Login Workflow
Admin	Views organization dashboard, manages employees, attendance reports, visitors, settings, announcements, and approvals.
HR	Views HR dashboard, manages people and operational workflows, handles attendance/ reporting, visitors, announcements, and assigned approvals.
Employee	Views personal dashboard, marks attendance, checks history, applies leave/permission/OD, reads announcements, and manages profile.
DRI	Views DRI dashboard, marks attendance, raises requests, and handles assigned request pages where permitted.

5. How the App Was Created From Scratch

The project was built as a full-stack JavaScript application with a Vite React frontend and an Express backend. The frontend focuses on screens, role-based navigation, responsive UI, mobile support, exports, and user interaction. The backend focuses on secure APIs, database models, business rules, report generation, notifications, uploads, and deployment configuration.

- Initial setup: a root project was created with separate client and server folders and convenience scripts for development and build commands.
- Frontend creation: React pages, layouts, protected routes, Tailwind styling, shared UI components, PWA files, and Capacitor Android setup were added.
- Backend creation: Express server, route modules, controllers, middleware, MongoDB models, SQL support modules, uploads, health endpoint, and production startup script were added.
- Database connection: Mongoose connects to MongoDB through MONGO_URI, with optional SQL Server/MySQL support present in the server/sql and database folders.
- Authentication setup: login, register, profile, change password, forgot password, reset password, JWT token generation, and route protection were implemented.
- UI design and theme: Tailwind CSS, HYA/Razk color styling, dark mode support, responsive layout, dashboards, status badges, cards, and chart colors were configured.
- Testing and debugging: build commands, server syntax checks, health checks, Render/Vercel deployment settings, mobile PWA install support, and Android APK build support were used during development.
- Deployment-ready configuration: Vercel SPA rewrite, Render startup script, Node engine selection, environment variable validation, CORS origin handling, and PWA manifest/service worker were added.

6. Technology Stack Used

Layer	Technologies Used
Frontend	React 18, Vite, React Router, Tailwind CSS, Lucide React, Recharts, Axios, React Hot Toast.
Backend	Node.js, Express, Mongoose, MongoDB, JWT, bcryptjs, Multer, Helmet, CORS, Morgan, express-rate-limit.
Reports	PDFKit for PDF reports and ExcelJS for Excel reports.
Database	MongoDB through Mongoose models. SQL Server/MySQL support files and schemas are also present.
Authentication	JWT bearer tokens, bcrypt password hashing, protected routes, and role-based API authorization.
Mobile/PWA	Capacitor Android, PWA manifest, service worker, installable web app support, Filesystem/Share plugins for mobile report handling.
Deployment Support	Vercel frontend configuration, Render backend startup script, Node 22 engine, environment variables, CORS origin handling.

7. Folder and File Structure Explanation

Path	Purpose
client/src/App.jsx	Defines frontend routes, lazy-loaded pages, protected routes, and role-based route groups.

Path	Purpose
client/src/layouts	Contains the main application layout and protected route wrapper.
client/src/pages	Contains app screens such as login, dashboards, employees, attendance, reports, requests, announcements, settings, and visitors.
client/src/components	Reusable UI components such as buttons, cards, badges, notification bell, company logo, forms, loading states, and dashboard charts.
client/src/context/ AuthContext.jsx	Manages login state, session storage token, logout, and authenticated user data.
client/src/services/api.js	Axios API client that attaches JWT tokens and handles API errors.
client/src/config/api.js	Selects correct backend API URL for local web, production web, and mobile builds.
client/public	Public logo, PWA icons, manifest, and service worker.
client/android	Capacitor Android project for APK/mobile app builds.
server/server.js	Main Express application, security middleware, CORS, health endpoint, route mounting, database startup, and error handling.
server/routes	API endpoint definitions grouped by feature.
server/controllers	Business logic for auth, employees, attendance, requests, reports, notifications, visitors, uploads, and settings.
server/models	MongoDB/Mongoose schemas for users, attendance, requests, announcements, notifications, office locations, and visitors.
server/middleware	Authentication, authorization, upload handling, and centralized error handling.
server/utils	Shared helpers for reports, Excel, dates, week-off logic, shifts, request workflow, geolocation, CSV, and default admin setup.
server/sql and server/database	Optional SQL Server/MySQL support modules, schemas, migrations, and documentation.
docs	Project documentation and generated explanation PDF.

8. Security and Best Practices

- Passwords are hashed with bcrypt before they are saved in the database.
- JWT bearer tokens protect private API routes, and the backend verifies tokens before allowing access.
- Role-based authorization restricts sensitive operations such as employee management, approvals, reports, visitor management, and office settings.

- Environment variables are used for database connection strings, JWT secrets, ports, and frontend origins.
- Helmet, CORS, and express-rate-limit are configured on the Express backend for baseline API hardening.
- Centralized async error handling is used through asyncHandler and error middleware.
- Frontend protected routes redirect unauthorized users to login.
- Uploads are handled through protected routes and organized by upload folder type.
- Attendance includes validation for duplicate check-in, check-out sequence, week-off rules, and working-hour calculations.
- The report API checks role access, especially limiting normal employees to their own attendance report.

9. Deployment and Installation Support

- The frontend can be built with npm run build inside the client folder and deployed as a static Vite application.
- client/vercel.json contains a SPA rewrite so direct route refreshes work correctly on Vercel.
- The backend can be installed from the server folder and started with npm start, which runs scripts/renderStart.js.
- The backend health endpoint reports server status, database status, environment configuration, and allowed origins.
- The PWA manifest and service worker allow users to install the web app on mobile and Windows browsers.
- Capacitor Android configuration supports building an Android APK from the same frontend build output.

10. Final Summary

Razk Automation Employee Management System is a complete business application for attendance and HR operations. It combines a secure role-based frontend, a structured backend API, MongoDB data storage, attendance workflows, request approvals, reports, announcements, notifications, visitor tracking, and mobile/PWA support.

The project provides practical value by replacing scattered manual processes with a single connected system. Admin and HR users can manage staff, approvals, and reports, while employees can complete daily attendance and request tasks from a browser, installed PWA, or Android app.

Future improvements could include stronger production monitoring, automated backups, advanced analytics, deeper audit trails, performance indexing for larger teams, queue-based report generation, and a paid hosting/database tier for higher user volume. Overall, the application is well-positioned as a professional internal HRMS and attendance management solution.

Assumption: This document describes the current project code and configuration available in the workspace. Runtime secrets, live hosting account settings, and production data values are not included.